1. Foundations & resource group
2. Azure SQL database
3. The real-money incident
4. App Service plan & web app
5. Managed identity & passwordless SQL
6. Schema via EF Core migrations
7. Entra ID app registrations
8. App configuration
9. Code: JWT auth in Program.cs
10. Deploy
11. The missing-user outage
12. Architecture, final

01 Foundations & resource group

Every Azure resource lives inside a resource group — the folder that groups things you'll want to delete, bill, or manage together.

CONCEPT

A **resource group** is a logical container, not a region or a server. Resources inside it can span regions. Deleting the group deletes everything in it — which is also the fastest way to guarantee a learning project stops costing anything.

```
az group create --name catalog-rg --location eastus
```

Later resources were created in `australiaeast` instead (see Problem below), but the group itself stayed in `eastus` — a group's own location is just metadata for where its deployment history is stored, it doesn't constrain where its resources run.

RESOURCE	NAME
Resource Group	`catalog-rg`

02 Azure SQL database

A logical SQL Server (the gatekeeper and auth boundary) plus a serverless database underneath it.

CONCEPT

In Azure SQL, the **server** isn't a VM — it's a management and authentication boundary (one admin login, one firewall, one Entra AD admin). The actual data and compute live in a **database** underneath it, and one server can hold many databases. The **serverless** compute tier auto-pauses the database after a period of inactivity and resumes it on the next connection, which is what makes near-zero-usage learning projects cheap.

PROBLEM 1 — SUBSCRIPTION NOT REGISTERED

ERROR

`MissingSubscriptionRegistration` on the very first `az sql server create`.

WHY IT HAPPENED

Azure requires each subscription to explicitly "register" a resource provider (like `Microsoft.Sql`) before it can create resources of that type. A fresh subscription has almost nothing registered.

FIX

```
az provider register --namespace Microsoft.Sql
```

PROBLEM 2 — PERSONAL ACCOUNT CAN'T BE AN AD-ONLY ADMIN

ERROR

`InvalidExternalAdministratorSid`, then `Resource 'chamodsr@gmail.com' does not exist or one of its queried reference-property objects are not present.`

WHY IT HAPPENED

The original plan was AD-only authentication (no SQL password at all) with the Gmail-based Microsoft account as the server's Entra admin. `az ad user show` looks up users in the tenant's directory by object type **user** — but a personal Microsoft account signed into a work/school tenant is represented differently and doesn't resolve the same way, so the Object ID lookup needed for `--external-admin-sid` failed.

FIX

Created the server with traditional SQL authentication (`--admin-user` / `--admin-password`) instead, and added an Entra admin *after* creation via the Portal, which resolves personal accounts correctly through the sign-in flow rather than a directory lookup.

PROBLEM 3 — REGION REJECTED PROVISIONING

ERROR

`RegionDoesNotAllowProvisioning` for East US; then a second attempt hit `InvalidResourceLocation` because a same-named server already half-existed there.

WHY IT HAPPENED

Azure occasionally throttles new SQL logical server creation in specific regions for specific subscription types (capacity management on Microsoft's end, not something you configure). Server names are also globally unique across all of Azure, so a failed attempt can still "reserve" a name.

FIX

Switched region to `australiaeast` (closer to home anyway) and used a new, unused server name: `sql-catalog-prod`.

PROBLEM 4 — SERVERLESS FLAGS ARE PICKY

ERROR

Multiple rejected flag combinations: edition/compute-model shorthand rejected, `-c 0.5` rejected (`ValueError: invalid literal for int()`), then `-c 1` alone matched a provisioned (non-serverless) SKU and errored.

WHY IT HAPPENED

Azure SQL vCore pricing needs edition (`-e`), hardware family (`-f`), capacity (`-c`), *and* an explicit `--compute-model Serverless` flag together — leaving any one out makes the CLI guess a different (usually provisioned, non-free-eligible) SKU that happens to share the same capacity number.

FIX

```
az sql db create \
  --resource-group catalog-rg --server sql-catalog-prod \
  --name CatalogDb \
  -e GeneralPurpose -f Gen5 -c 1 \
  --compute-model Serverless \
  --auto-pause-delay 60
```

RESOURCE	NAME / VALUE
SQL Server	`sql-catalog-prod.database.windows.net` (australiaeast)
Database	`CatalogDb` — GeneralPurpose, Serverless, Gen5, 1 vCore, 60-min auto-pause
Entra admin	`chamodsr` (added via Portal)
Firewall	`AllowMyIP`, `AllowAzureServices` (0.0.0.0)

03 The real-money incident

The one phase every future Azure learning project should read first.

INCIDENT

AU$2.26 in genuine charges appeared on the account once the 30-day trial credit (A$0.00 remaining) ran out.

WHY IT HAPPENED

A `GeneralPurpose Serverless` Azure SQL database is **not free by default** — it's billed per vCore-second of active compute plus storage, auto-pause or not. Azure does offer

one database per subscription under an "Always Free" limit, but it only applies when the database is explicitly created with that flag. The database in Phase 2 was created without it, so every query (including the app's own health checks and migrations) was quietly metered against the trial credit, then against the card on file once that credit hit zero.

CONCEPT — AZURE "ALWAYS FREE" VS. THE 30-DAY TRIAL

These are two different programs. The 30-day trial gives a pool of credit (here, converted to A$0.00 once exhausted) that offsets *any* usage. "Always Free" is a separate, permanent allowance — one small SQL database, one small App Service plan, etc. — that costs nothing **indefinitely**, trial or no trial, but only for resources created with the specific flag that opts into it.

FIX

Deleted and recreated `CatalogDb` with the free-limit flags:

```
az sql db create \
  --resource-group catalog-rg --server sql-catalog-prod \
  --name CatalogDb \
  -e GeneralPurpose -f Gen5 -c 1 \
  --compute-model Serverless \
  --auto-pause-delay 60 \
  --use-free-limit \
  --free-limit-exhaustion-behavior AutoPause
```

Verified via the response JSON: `"useFreeLimit": true`, `"kind": "...,freelimit"`. `AutoPause` means that if the free allowance is ever fully used in a month, the database simply pauses rather than silently billing — it fails safe instead of failing expensive.

As a second safety net (not a substitute for the fix above), a Cost Management budget alert was also set up: `catalog-learning-budget`, $1/month, 80% threshold, emailing `chamodsr@gmail.com`. The CLI form of this command (`az consumption budget create`) kept failing on multi-line JSON notification arguments getting mangled by the terminal — it was created through the Portal's Cost Management → Budgets screen instead.

---

04 App Service plan & web app

The compute layer that actually runs the API.

CONCEPT

A **App Service Plan** is the underlying compute (CPU/RAM/SKU) — think of it as the VM pool. A **Web App** is the actual application that runs on that plan. Multiple web apps can share one plan. The free SKU is `F1`, nicknamed `LinuxFree` on Linux plans.

PROBLEM — PAID SKU PROPOSED BY DEFAULT

CORRECTION

The first plan suggested `B1` (Basic tier — paid, ~$13/month), which conflicted directly with the stated goal of a zero-cost learning project.

FIX

```
az appservice plan create \
  --resource-group catalog-rg --name catalog-plan \
  --is-linux --sku F1
```

This briefly appeared to fail with a Linux-incompatibility error — that turned out to be a red herring. The real cause was that `Microsoft.Web`, like `Microsoft.Sql` in Phase 2, hadn't auto-registered yet on the subscription. Once it registered, the identical command succeeded with a genuine `"tier": "LinuxFree"` in the response — F1 does support Linux.

RESOURCE	NAME / VALUE
App Service Plan	`catalog-plan` — Linux, F1 (LinuxFree)
Web App	`catalog-api-chamod.azurewebsites.net`, runtime `DOTNETCORE:9.0`
Identity	System-assigned managed identity, principal ID `8acb0332-c310-44ef-aed5-bf153dd414c1`

05 Managed identity & passwordless SQL

Letting the Web App prove who it is to Azure SQL without a password anywhere in config.

CONCEPT — MANAGED IDENTITY

A **system-assigned managed identity** is an auto-created, auto-rotated Entra ID identity tied to one specific Azure resource's lifecycle. The app can ask Azure's internal metadata endpoint for a short-lived access token for any service that trusts Entra ID — Azure SQL included — with zero secrets stored anywhere. Inside the container this shows up as two environment variables: `IDENTITY_ENDPOINT` and `IDENTITY_HEADER`, which the SDK uses to fetch a token locally without a round-trip to sign in interactively.

Azure SQL needs to know that identity by name. That's a **contained database user** — a login that exists only inside one database, mapped to an external (Entra ID) principal rather than a SQL password:

```
CREATE USER [catalog-api-chamod] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [catalog-api-chamod];
ALTER ROLE db_datawriter ADD MEMBER [catalog-api-chamod];
```

The connection string then names the auth mode instead of a password:

```
Server=tcp:sql-catalog-prod.database.windows.net,1433;Database=CatalogDb;Authentication=Activ
```

`Active Directory Default` tells the SQL client driver to use the same credential-discovery chain as `DefaultAzureCredential` — in Azure, that resolves to the managed identity automatically; run locally, it falls back toward your own signed-in CLI session instead.

---

06 Schema via EF Core migrations

The existing migration was generated for SQLite. SQL Server needed its own.

PROBLEM — STALE ENV VARS ACROSS TERMINAL TABS

ERROR

The regenerated migration still produced SQLite-flavored types (`TEXT` for a `decimal` column) instead of SQL Server types (`decimal(18,2)`).

WHY IT HAPPENED

`Database__Provider` and `ConnectionStrings__CatalogDb` were exported in one terminal tab, but `dotnet ef migrations add` was run in a different tab/session — environment variables set with `export` don't persist across separate terminal windows

or new shell sessions.

FIX

Re-exported both variables in the exact terminal used to run the EF command, confirmed with `echo $Database__Provider` before regenerating — the migration then correctly produced `nvarchar`, `decimal(18,2)`, and `datetime2`.

NOTE — DESIGN-TIME VS. RUNTIME

`dotnet ef migrations add` never connects to a real database — it inspects the C# model at design time and writes migration files. No sign-in prompt is expected there. The actual connection (and any AD sign-in) happens at `dotnet ef database update`, which applies the migration against the live database.

---

07 Entra ID app registrations

Two registrations: one representing the API, one representing the Postman client calling it.

CONCEPT — API VS. CLIENT REGISTRATION

The **API registration** (`CatalogApi`) defines what the API *is*: its audience (`api://<client-id>`) and the scopes callers can request, e.g. `access_as_user`. The **client registration** (`CatalogApi-Postman`) represents a caller — it's "pre-authorized" against the API's scope so Postman can request a token for that audience without an admin consent screen interrupting the flow. Supporting personal Microsoft accounts *and* work/school accounts means setting `signInAudience` to `AzureADandPersonalMicrosoftAccount` and keeping `TenantId` as `common` rather than a specific tenant GUID, with the token version forced to v2 (`requestedAccessTokenVersion: 2`) so claims are shaped consistently across both account types.

REGISTRATION	CLIENT ID	ROLE
`CatalogApi`	`ec99e890-c673-46c7-8d84-cc3db40275af`	API — exposes `access_as_user` scope
`CatalogApi-Postman`	`3cdaf348-c5f0-4563-bdea-1382a9f5be26`	Public client — pre-authorized caller

Tenant: `926d8da7-8aa9-434d-8f6b-151f121a09f9`. Audience: `api://ec99e890-c673-46c7-8d84-cc3db40275af`.

08 App configuration

Wiring the Web App's environment to point at SQL Server, not SQLite, and to trust the right tenant.

SETTING	VALUE
`ASPNETCORE_ENVIRONMENT`	`Production`
`Database__Provider`	`SqlServer`
`Database__ApplyMigrationsOnStartup`	`false` — migrations are applied by hand, not on every cold start
`ConnectionStrings__CatalogDb`	passwordless, `Authentication=Active Directory Default`
`AzureAd__Instance` / `__TenantId` / `__ClientId`	points JWT validation at the `CatalogApi` registration

PROBLEM — CLI SHOWS EVERY VALUE AS NULL

NOT ACTUALLY A BUG

`az webapp config appsettings list` returned `"value": null` for every setting after setting them.

WHY IT HAPPENED

Azure CLI has redacted app-setting values from `list` / `set` output since ~2023, as a security measure against secrets leaking into terminal scrollback or CI logs. The settings were in fact saved correctly.

FIX

Verified real values two ways instead: the Portal's Environment variables blade (screenshot confirmed all settings present), and later, directly inside the running container via `az webapp ssh` → `printenv`.

PROBLEM — WRONG FLAG FOR HEALTH-CHECK PATH

ERROR

`az webapp config set --health-check-path /health/ready` → `unrecognized arguments`.

WHY IT HAPPENED

That flag doesn't exist on `config set` in this CLI version; health check path is applied as a site configuration value, not a dedicated flag.

FIX

```
az webapp config set \
  --resource-group catalog-rg --name catalog-api-chamod \
  --generic-configurations '{"healthCheckPath": "/health/ready"}'
```

Confirmed afterwards with `az webapp config show --query "{healthCheckPath: healthCheckPath}"`.

---

09 Code: JWT auth in Program.cs

The one phase with real code changes — written by hand, reviewed line by line before saving.

Added `Microsoft.Identity.Web`, wired up JWT bearer validation against the `AzureAd` config section, and protected the product endpoints while leaving health checks open to anonymous probes:

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApi(builder.Configuration.GetSection("AzureAd"));
builder.Services.AddAuthorization();

// ...

app.UseAuthentication();
app.UseAuthorization();
app.MapProductEndpoints(); // .RequireAuthorization() added on the group
```

PROBLEM — SQL SERVER MIGRATION BREAKS LOCAL SQLITE DEV

ERROR

`SQLite Error 19: 'NOT NULL constraint failed: Products.Id'` during local seed data insert, after the SQL-Server-targeted migration was in place.

WHY IT HAPPENED

The migration (generated in Phase 6 for SQL Server) used `nvarchar`, `decimal(18,2)`, `datetime2` — SQLite silently accepts that syntax without erroring, but SQLite's auto-increment rowid alias only activates on a column declared with the *exact* type `INTEGER`. The SQL Server migration's `int` type doesn't qualify, so `Id` was never auto-populated, and the insert failed its `NOT NULL` check.

FIX

Branched schema creation by provider in `Program.cs`: `MigrateAsync()` for SQL Server (applies the real migration), `EnsureCreatedAsync()` for SQLite (builds the schema straight from the C# model, bypassing the mismatched migration entirely).

Test coverage grew from 16 to 17 cases with a fake auth handler (`TestAuthHandler`) standing in for real Entra tokens in integration tests — it succeeds whenever any `Authorization` header is present, letting tests simulate both authenticated and unauthenticated calls without a live tenant.

---

10 Deploy

From a published build to a running container, in three commands.

```
dotnet publish src/CatalogApi/CatalogApi.csproj -c Release -o ./publish
cd publish && zip -r ../publish.zip . && cd ..
az webapp deploy \
  --resource-group catalog-rg --name catalog-api-chamod \
  --src-path publish.zip --type zip
```

Result: `Deployment has completed successfully`, `"status": "RuntimeSuccessful"`. For a pre-built zip like this, no build step runs server-side — that's why the deploy only takes ~50 seconds instead of several minutes.

11 The missing-user outage

A proper little debugging story — five layers peeled back in order.

SYMPTOM

`/health` (liveness, no DB touch) returned 200 instantly. `/health/ready` (DB check) hung for 60–100 seconds, then returned `503 Unhealthy` with no exception message — just `'(null)'` in the logs.

ELIMINATION, STEP BY STEP

1. **Firewall rules** — checked with `az sql server firewall-rule list`. Both `AllowAzureServices` and `AllowMyIP` were present and correct. Ruled out.
2. **Connection string value** — CLI listing is redacted (see Phase 8), so verified by SSHing into the container and reading the live environment directly (`az webapp ssh` → `printenv | grep catalogdb`). The string was correctly formed. Ruled out.
3. **Managed identity presence** — confirmed `IDENTITY_ENDPOINT` and `IDENTITY_HEADER` existed in the container. Ruled out.
4. **Raw network path** — tested with a bare TCP probe from inside the container: `echo > /dev/tcp/sql-catalog-prod.database.windows.net/1433` → succeeded. Ruled out.
5. **Token acquisition** — called the managed identity endpoint directly with `curl` and got a valid access token back instantly, with an `oid` claim matching the Web App's own managed identity principal ID. Ruled out.

Everything up to and including authentication worked. The remaining candidate was **SQL-side authorization** — querying `sys.database_principals` in the Portal's Query Editor confirmed it: **zero rows** for the managed identity's contained user.

WHY THE USER HAD VANISHED

The contained database user created back in Phase 5 lived inside the *original* `CatalogDb` — the one deleted and recreated during the Phase 3 cost incident. Deleting a database deletes everything inside it, contained users included. The free-limit database that replaced it was structurally identical but had none of its own users yet. EF Core migrations had already been re-applied against it (via the Entra admin account, which doesn't need a contained user), so the schema looked complete — only the managed identity's own access was missing, and nothing about schema state would have revealed that.

FIX

```
CREATE USER [catalog-api-chamod] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [catalog-api-chamod];
ALTER ROLE db_datawriter ADD MEMBER [catalog-api-chamod];
```

`/health/ready` returned `200 Healthy` on the next request. `/api/products` without a token correctly returned `401 Unauthorized` with a `WWW-Authenticate: Bearer` header.

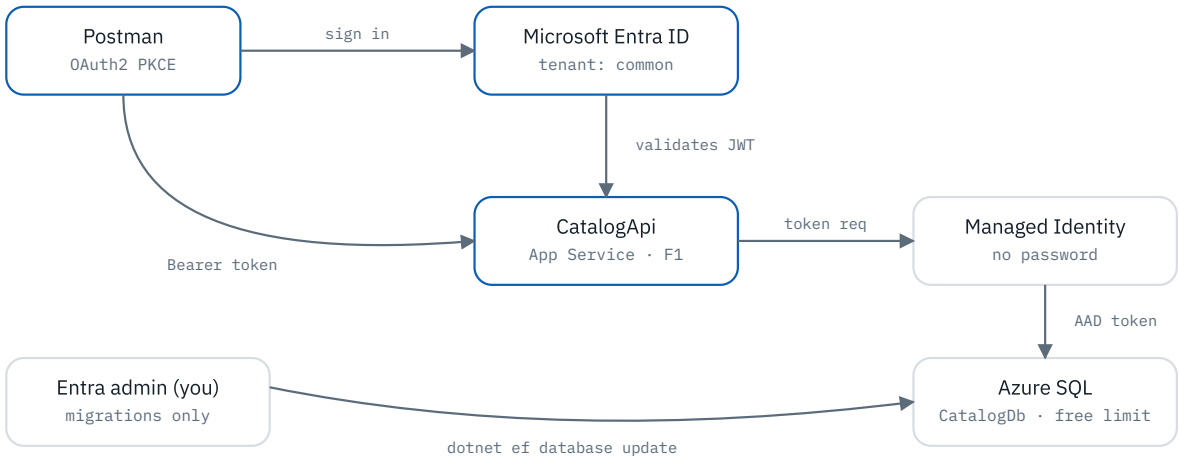
LESSON

Recreating a database resets its principals even when the schema and server around it look unchanged. Any time a database is deleted and rebuilt — for cost reasons or otherwise — contained users for managed identities need to be recreated too; they're not part of the schema and migrations won't restore them.

---

12 Architecture, final

Everything built, end to end, zero passwords in configuration.



Two separate identities touch the database for two different reasons: your own Entra admin account applies schema migrations by hand (deliberately — `ApplyMigrationsOnStartup` is `false` in production), while the Web App's managed identity only ever reads and writes rows through `db_datareader` / `db_datawriter`, with no path to altering schema.

What this project actually taught

- Azure's free tiers are opt-in flags, not defaults — `--use-free-limit`, `F1` / `LinuxFree`, each has to be asked for explicitly, and the ordinary SKU one layer up is usually the one that gets created by accident.
- Managed identity removes passwords from config, but it doesn't remove the need for an explicit grant — a contained database user still has to exist, and it's scoped to one specific database's lifetime, not the server's.
- When a symptom is silent (a null exception message, a value that reads as redacted rather than wrong), isolate the system boundary by boundary — network, then identity, then authorization — rather than guessing at the whole stack at once.
- Environment variables set with `export` are a property of one terminal session. Nothing about EF Core or Azure CLI carries them anywhere else.

`CatalogApi · catalog-rg · australiaeast — compiled from the deployment session, 2026-10-10/11`